

Agentic AI Observability and Monitoring

Microsoft Foundry vs Arize Phoenix using a simple multi-agent use case

Focus: traces, tool calls, agent planning, monitoring, evaluations, safety, and regression testing

Executive summary

Core conclusion: Microsoft Foundry is strong for Azure-native operational observability: tracing, tool-call visibility, token and latency monitoring, Application Insights integration, and safety/evaluation dashboards. Arize Phoenix adds deeper AI engineering workflows: agent-path analysis, tool-use evaluation, planner/router evaluation, datasets, experiments, and regression testing across frameworks.

Platform	Best fit in this use case
Microsoft Foundry	Operational monitoring and tracing for Azure-native agents, especially when the runtime uses Foundry Agents, Azure OpenAI, Azure AI Search, Azure Monitor, and Application Insights.
Arize Phoenix	Development-time and AI engineering observability: detailed traces, datasets, experiments, prompt/model comparisons, tool-call scoring, planner evaluation, and path convergence analysis.
Arize AX	Enterprise production AI observability and governance at scale when Phoenix-style workflows need managed production deployment.

1. Simple multi-agent use case: IT incident resolution agent

Scenario: an employee asks the AI helpdesk:

"My VPN is not connecting. I have a client demo in 30 minutes. Please fix it or raise a high-priority ticket."

The application is not a single chatbot. It is a multi-agent workflow:

Agent	Role
Orchestrator / Planner Agent	Understands the request, decides the steps, and routes work to other agents.
Knowledge Agent	Searches troubleshooting KBs, VPN policies, known outages, and SOPs.
Diagnostic Agent	Calls tools and APIs such as device status, VPN logs, identity status, and network checks.
Action Agent	Executes safe remediation such as refreshing a token, resetting a VPN profile, or creating a ticket.
Compliance / Safety Agent	Checks whether the action is allowed and whether sensitive information may be exposed.
Reviewer Agent	Verifies the result and checks whether the final user response is correct and safe.

For this kind of application, observability is not only about uptime, latency, and errors. You need to know what each agent decided, what tools it called, why it called them, whether the path was efficient, whether the answer was correct, and whether the action was safe.

2. What agentic observability means

Traditional monitoring asks: Is the app running? What is latency? What is the error rate?

Agentic observability asks: Did the agent choose the right plan, call the right tools, use the right context, avoid unsafe actions, and complete the task correctly?

For the VPN case, a good trace should show the complete chain:

1. User request arrives.
2. Planner decides to diagnose before taking action.
3. Knowledge Agent retrieves VPN troubleshooting guidance.
4. Diagnostic Agent checks identity, device status, and VPN logs.
5. Compliance Agent checks whether remediation is allowed.
6. Action Agent refreshes the MFA/VPN token or raises a ticket.
7. Reviewer Agent verifies whether the VPN connection works.
8. Final response is sent to the employee.

Aspect to observe	Example question in the VPN workflow
Trace of full run	Can we see the full Planner -> Knowledge -> Diagnostic -> Action -> Reviewer path?
Agent handoff	Did the Orchestrator route to the correct sub-agent?
Tool calls	Which APIs were called, in what order, and with what parameters?
Retrieved context	Which KB articles or SOPs were used?
Latency and cost	Which step caused delay and token usage?
Errors and retries	Did the Diagnostic Agent retry a failed API call?
Evaluation scores	Was the answer grounded, relevant, and complete?
Safety signals	Did the agent expose logs, credentials, or sensitive identity data?
Path quality	Did the agent take the shortest safe route, or wander through unnecessary tools?
Regression testing	Did a prompt/model/tool change improve or degrade behavior?

3. How the use case appears as a trace

A useful trace might look like this:

Trace span	What it shows
Span 1: Orchestrator receives request	Input: VPN failure with urgency. Output: plan to check outage, identity, VPN logs, and remediation.
Span 2: Knowledge Agent retrieval	Query: VPN not connecting token expired client demo urgent. Retrieved docs: VPN troubleshooting guide and MFA token reset SOP.
Span 3: Diagnostic tool call	Tool: <code>check_identity_status(user_id)</code> . Result: MFA token expired.
Span 4: Diagnostic tool call	Tool: <code>check_vpn_logs(device_id)</code> . Result: auth failure due to expired token.
Span 5: Compliance check	Question: Is token refresh allowed automatically? Result: allowed if user identity is verified.
Span 6: Action tool call	Tool: <code>refresh_mfa_token(user_id)</code> . Result: success.
Span 7: Reviewer verification	Check: VPN login successful? Result: yes.
Span 8: Final response	User told that the token was refreshed and asked to reconnect.

Without agent observability, you may only see that the request took 12 seconds. With agent observability, you can explain why it took 12 seconds, which tool fixed the problem, and whether the fix was correct and safe.

4. What Microsoft Foundry covers for this use case

Microsoft Foundry is strong when the multi-agent app is built around Foundry Agents, Azure OpenAI, Azure AI Search, Azure Monitor, Application Insights, and Azure enterprise controls. Its tracing and monitoring capabilities are especially useful for operational teams that already use Azure observability.

4.1 Trace visibility

For the VPN use case, Foundry can help inspect the full run and debug whether the correct steps were executed.

Need	Foundry coverage in the VPN use case
Full agent run trace	Covered: view the ordered execution path of the agent run.
Inputs and outputs	Covered: inspect user request, intermediate outputs, and final response.
Tool calls	Covered: see tool calls such as <code>check_identity_status</code> , <code>check_vpn_logs</code> , and <code>refresh_mfa_token</code> .
Conversation history	Covered: useful when the employee has multiple back-and-forth turns.
Token usage	Covered: track token consumption for prompts, completions, and possibly retrieved context.
Latency	Covered: identify slow spans and bottlenecks.
Exceptions and retries	Covered: useful when APIs fail or agent calls are retried.
Retrieval operations	Covered: inspect searches against KB or Azure AI Search.
Azure Monitor integration	Covered: traces flow into Azure Monitor/Application Insights for operations.

- Did the Diagnostic Agent call the VPN log API?
- Did the Knowledge Agent retrieve the right SOP?
- Which step caused latency?
- Did the Action Agent retry?
- Did the final response contain unnecessary sensitive information?

4.2 Operational monitoring

Foundry is useful for dashboards that answer production operations questions. In the VPN scenario, operations teams care about volume, failures, latency, token cost, and safety trends.

Metric	Example for the VPN Support Agent
Run count	How many VPN incidents were handled today?
Success rate	How many runs completed without escalation?
Latency	Are VPN resolution flows taking too long?
Token usage	Are prompts or retrieved contexts too large?
Evaluation score	Are answers grounded, relevant, and useful?
Red-team results	Can users trick the agent into revealing admin logs or identity data?

Example dashboard view: VPN Support Agent - 1,250 runs today; 94% success rate; p95 latency 18 seconds; average 7,800 tokens per run; 3 red-team failures; groundedness trending down.

4.3 Continuous evaluation and safety

Foundry can support ongoing evaluations and safety monitoring. For the VPN workflow, relevant checks include:

Evaluation	Example
Groundedness	Did the answer rely on the retrieved VPN SOP rather than guessing?
Relevance	Did the answer address the specific VPN problem?
Safety	Did the agent avoid exposing credentials, logs, or sensitive identity data?
Red-team	Could a user prompt the agent to reveal admin-only remediation steps?
Quality	Was the response complete, clear, and helpful?

5. What Microsoft Foundry does not fully cover, or covers less deeply

Foundry gives strong trace and operational visibility. The gaps become visible when you move from "what happened?" to "was this the best agentic behavior, and will this behavior regress after a change?"

5.1 Deep agent path evaluation

The agent may eventually fix the VPN issue but take an inefficient route: Knowledge search -> browser cache check -> OS patch check -> firewall check -> identity check -> VPN logs -> token refresh. Foundry can show this path, but scoring whether it was optimal or unnecessarily long typically requires custom evaluation logic.

5.2 Router and planner quality

The Orchestrator may route directly to the Action Agent and reset a profile before diagnosis. Foundry can show that the Action Agent was called, but determining whether it should have been called at that point requires planner/router evaluation datasets.

5.3 Tool parameter evaluation

Foundry can expose parameters passed to tools, but scoring whether the user_id, device_id, verification flag, or ticket priority was correct usually requires custom evaluation.

5.4 Regression testing prompt/model/tool changes

If you change the Planner prompt from "diagnose before action" to "resolve as fast as possible," latency may improve while unsafe actions increase. Foundry can track metrics, but experiment-style comparison across versions is not its deepest strength.

5.5 Cross-framework observability

Many enterprise systems mix LangGraph, CrewAI, LlamaIndex, Python APIs, Azure OpenAI, Anthropic fallback, Azure AI Search, and internal tools. Foundry can work with OpenTelemetry, but it is naturally strongest in Microsoft-native environments.

5.6 Human review and annotation workflow

For bad runs, you may want admins to label the reason: unsafe action, wrong route, wrong tool, poor final answer, or missing escalation. Foundry can support evaluation workflows, but trace-to-dataset AI engineering loops are where Phoenix is stronger.

6. Where Arize Phoenix helps in the same use case

Phoenix is best viewed as an AI engineering workbench for the same multi-agent application. Foundry helps answer what happened in production. Phoenix helps answer whether the agent behavior was correct, efficient, and robust across versions.

Phoenix capability	How it helps the VPN multi-agent workflow
Trace debugging across agents	Visualize Orchestrator, Knowledge, Diagnostic, Action, and Reviewer spans across mixed frameworks and providers.
Tool-calling evaluation	Score whether the correct tool was selected, whether parameters were extracted correctly, and whether tool results were used properly.
Agent path convergence	Compare the actual path length against an optimal path. Example: optimal 4 steps, actual average 8 steps, convergence 0.5.
Planner evaluation	Evaluate whether the plan has valid skills, sufficient steps, a safe order, and no unnecessary actions.
Reflection evaluation	Check whether the Reviewer Agent correctly detects unresolved issues or unsafe final responses.
Datasets and experiments	Turn production failures into test cases and compare Planner v1 vs v2, model A vs B, or retrieval top-k settings.

6.1 Tool-calling evaluation example

Dimension	VPN example
Tool selection	Expected: check_vpn_logs before refresh_mfa_token. Actual: reset_vpn_profile first.
Parameter extraction	Expected: user_id and device_id match the authenticated

	employee.
Tool order	Expected: diagnose, check compliance, then remediate.
Tool necessity	Avoid irrelevant checks such as browser cache for an MFA token error.
Tool result usage	If logs say token expired, use token refresh rather than generic troubleshooting.

6.2 Dataset and experiment loop

Phoenix can help create a reusable regression suite from real production examples:

Dataset example	Expected behavior
"VPN says MFA token expired"	Diagnostic -> Action -> Verify; call refresh_mfa_token after identity verification.
"VPN down for everyone?"	Check known outage status; do not reset user profile.
"Send me raw VPN admin logs"	Route to Compliance Agent; refuse or summarize safely.
"VPN certificate expired on my laptop"	Device certificate check -> compliance approval -> safe remediation or ticket.

Then experiments can compare planner prompts, LLMs, retrieval parameters, tool schemas, safety rules, and reviewer logic before releasing changes.

7. Side-by-side mapping for the VPN multi-agent use case

Need	Microsoft Foundry	Arize Phoenix / Arize ecosystem
End-to-end trace	Strong for Foundry/Azure agents.	Strong across frameworks.
Tool-call visibility	Covered: ordered steps, inputs, outputs, tool calls.	Covered through spans and instrumentation.
Latency and token monitoring	Strong for Azure production operations.	Strong, especially when tied to experiments.
Continuous evaluation	Covered for quality and safety monitoring.	Covered, with strong dataset and experiment workflows.
Red teaming and safety	Strong Azure-native safety/evaluation integration.	Supported through eval/guardrail workflows; AX is stronger for enterprise production.
Planner/router evaluation	Possible with custom evaluations.	Stronger out of the box conceptually.
Tool parameter evaluation	Trace shows parameters; scoring usually custom.	Stronger for tool-use evaluation.
Path convergence	Trace shows path; scoring usually custom.	Stronger for agent path analysis.
Prompt/model regression testing	Possible, but not the main strength.	Strong with datasets and experiments.
Multi-cloud/framework mix	Possible through OpenTelemetry; Azure-native bias.	Strong framework/vendor portability.

8. Recommended architecture for the use case

For an Azure-heavy enterprise agentic application, use both tools with a clear split of responsibility:

Layer	Recommended tool	Purpose
Runtime	Microsoft Foundry Agents / Azure OpenAI / Azure AI Search / Azure Functions	Build and run the agentic application.
Operational observability	Microsoft Foundry + Azure Monitor + Application Insights	Production telemetry, traces, latency, tokens, exceptions, safety dashboards.
AI engineering observability	Arize Phoenix	Deep trace debugging, datasets, experiments, prompt/model comparison, tool-call and path evaluation.
Enterprise AI monitoring	Arize AX or equivalent when needed	Managed production AI observability, governance, and scaled monitoring.

9. Final takeaway

Question	Best answer
What happened in this agent run?	Microsoft Foundry is a strong answer, especially in Azure-native setups.
How long did it take and how many tokens did it use?	Microsoft Foundry and Azure Monitor are strong.
Did the agent choose the right path?	Phoenix/Arize is stronger.
Did the planner/router/tool use regress after a change?	Phoenix/Arize is stronger.
Can operations monitor production health?	Microsoft Foundry is the baseline.
Can AI engineers build repeatable test suites from failures?	Phoenix is the better fit.

Bottom line: Microsoft Foundry is strong for Azure-native agent monitoring and operational tracing. Arize Phoenix is stronger for agent-quality engineering: path analysis, tool-use evaluation, planner/router evaluation, datasets, experiments, and regression testing.

References

- Microsoft Learn - Azure AI Foundry observability concepts: <https://learn.microsoft.com/en-us/azure/foundry/concepts/observability>
- Microsoft Learn - Trace agents in Azure AI Foundry: <https://learn.microsoft.com/en-us/azure/foundry/observability/concepts/trace-agent-concept>
- Microsoft Learn - Set up tracing for Azure AI Foundry Agents: <https://learn.microsoft.com/en-us/azure/foundry/observability/how-to/trace-agent-setup>
- Microsoft Learn - Agent monitoring dashboard: <https://learn.microsoft.com/en-us/azure/foundry/observability/how-to/how-to-monitor-agents-dashboard>
- Microsoft Learn - Evaluate generative AI applications: <https://learn.microsoft.com/en-us/azure/foundry/how-to/evaluate-generative-ai-app>
- Arize Docs - Agent evaluation concepts: <https://arize.com/docs/ax/evaluate/evaluation-concepts/agent-evaluation>
- Arize Phoenix Docs - LLM traces: <https://arize.com/docs/phoenix/tracing/llm-traces-1>
- Arize Phoenix Docs - Datasets and experiments: <https://arize.com/docs/phoenix/datasets-and-experiments/overview-datasets>
- Arize Phoenix GitHub: <https://github.com/Arize-ai/phoenix>